



République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
FACULTÉ
D'INFORMATIQUE
Université des Sciences et de la Technologie Houari Boumediene

Multimédia Project Report

M1 IL

Simplified MPEG-4 Video Encoder Pipeline

Realised by :

- OUALIHINE Douaa Chahrazed 212131088275
- LACHOURI Lyna 222231362304

1. Pipeline Description

We built a complete, simplified video compression pipeline inspired by MPEG-4. The input is a .y4m video file (flower_cif) from which we extract raw frames. Each frame goes through five stages before ending up in a compressed .bin file that can be decoded back into images.

Stage 0 — Frame Extraction

We used OpenCV to read the flower_cif.y4m file and extract individual frames. One frame was saved every 0.3 seconds of video, giving us a manageable set of images stored as .png files in a frames/ folder. This acts as the raw input to the rest of the pipeline.

Stage 1 — Pre-processing (Part 1)

Each BGR frame from OpenCV is converted to the YCbCr color space. Y carries the luminance (brightness), Cb carries the blue-difference chroma, and Cr carries the red- difference chroma. After conversion, we apply 4:2:0 chroma subsampling: Cb and Cr are downsampled by a factor of 2 in both width and height. This works because the human eye is much less sensitive to color detail than to brightness, so we can discard half the chroma information with barely any visible difference. The Y channel keeps its full resolution.

Stage 2 — Intra-frame Coding / I-frames (Part 2)

An I-frame is compressed entirely on its own, without looking at any other frame. We use the 2D Discrete Cosine Transform (DCT) on 8×8 pixel blocks. The DCT converts pixel values into frequency coefficients: the top-left coefficient (called the DC coefficient) represents the average brightness of the block, while the other coefficients (AC) represent finer details and edges.

After the DCT, we quantise the coefficients by dividing them element-wise by a quantisation matrix (inspired by the JPEG standard) and rounding to integers. This is where the lossy compression happens — small values become zero and are effectively discarded. High-frequency coefficients (fine details) are divided by larger numbers, so more information is lost there. A quality factor (1 to 100) scales the matrix: quality=100 means almost no loss, quality=1 means heavy compression. We used quality=50 as our default.

To decode an I-frame, we multiply the quantised coefficients back by the matrix (dequantise) and apply the inverse DCT (IDCT), then clip to [0,255]. All three channels (Y, Cb, Cr) are encoded and decoded independently.

Stage 3 — Inter-frame Coding / P-frames (Part 3)

Consecutive video frames are often very similar. Instead of compressing each one independently, we exploit this temporal redundancy.

We use a Group of Pictures (GOP) structure with `GOP_SIZE = 5`. Every 5th frame is coded as an I-frame (fully independent). All remaining frames are coded as P-frames, giving the following structure:

Frame position in GOP	Type	Reference
0	I-frame	None (fully independent)
1 – 4	P-frame	Previous reconstructed frame
5	I-frame	None (fully independent)
6 – 9	P-frame	Previous reconstructed frame

P-frames reference one past frame. The Y channel is divided into 16×16 macroblocks. For each macroblock, we search the previous reconstructed frame within a ± 8 pixel window using full search block matching with Sum of Absolute Differences (SAD). The result is a motion vector (dy , dx) indicating where the best match is in the reference frame. We then compute the residual (current block minus predicted block) and apply DCT + quantisation to that residual only. Storing the residual takes far fewer bits than storing the full block, because residuals are mostly near zero when the prediction is good.

The Cb and Cr channels of P-frames are coded with DCT + quantisation (like I-frames), since motion estimation is only applied to the Y channel.

The decoder reconstructs each P-frame by extracting the predicted block from the reference frame using the stored motion vector, decoding the residual (IDCT + dequantisation), and adding the two together.

Stage 4 — Entropy Coding (Part 4)

Once all frames are encoded, we serialize the data using Python's pickle module and compress it with zlib (level 6). This is the lossless layer on top of the lossy DCT compression. Motion vectors are stored as `int16` arrays to reduce the serialization footprint.

The output file starts with a magic header (MPEG4SIM), then each frame is stored as: 1 byte for the frame type (I or P), 4 bytes for the payload length, then the zlib-compressed pickle payload. A matching reader function reverses this process.

Stage 5 — Evaluation & Visualisation (Part 5)

We measured the compression ratio (raw YCbCr bytes / compressed .bin size).

The visualisation produces a single figure covering all stages: original frames, YCbCr channels, the DCT block pipeline, motion vectors overlaid on a P-frame, a residual heat map, and a size comparison bar chart.

2. Design Choices & Justification

Color space: YCbCr with 4:2:0 subsampling

YCbCr is the standard color space used in all real video codecs (MPEG, H.264, H.265). The key advantage is that separating luma from chroma lets us apply more compression to chroma without visible quality loss, because the human visual system resolves color much less sharply than brightness. 4:2:0 halves Cb and Cr in both dimensions, reducing chroma data by 75% with negligible perceptual impact.

8×8 DCT blocks for I-frames and residuals

8×8 is the block size used by JPEG and MPEG. It is a good balance: large enough to capture useful frequency structure, small enough to stay locally uniform. The DCT concentrates most of the signal energy in just a few low-frequency coefficients, so quantisation can zero out most high-frequency terms with minimal visible distortion.

16×16 macroblocks for motion estimation

Motion estimation works at a coarser scale than the DCT. 16×16 is the standard macroblock size for motion compensation in MPEG-4. Larger blocks are more efficient to search and produce smoother motion fields, at the cost of less precision for complex or fast-moving regions.

Full search with SAD (Sum of Absolute Differences)

We used an exhaustive search within a ± 8 pixel window to find the best matching block in the reference frame. SAD is fast to compute and works well in practice. A ± 8 window covers most motion in a slow-moving video like `flower_cif`. Larger windows would find better matches for fast motion but are much slower.

GOP size = 5 with I and P frames only

A GOP of 5 means one I-frame followed by 4 P-frames. This gives a good ratio of I-frames (expensive but self-contained) to P-frames (cheap, reference-based). Each P-frame references only the immediately preceding reconstructed frame, which keeps the decoder simple and straightforward: frames are always decoded in display order with no reordering needed.

Compared to a design that includes B-frames, this approach is simpler to implement and debug, and avoids the need for a two-pass encoder. The trade-off is slightly lower compression efficiency, since P-frames only use one reference instead of two.

Quality factor = 50

Quality 50 is the JPEG default and gives a reasonable trade-off. As shown in the experimental section, it produces a compression ratio around 4–5×. Lower quality

compresses more but introduces visible blocking artifacts; higher quality preserves detail but reduces compression efficiency.

Zlib for entropy coding

Rather than implementing Huffman or arithmetic coding from scratch, we used Python's built-in zlib library which applies DEFLATE compression. This gives genuine lossless compression on top of the quantised coefficients. The quantised DCT blocks contain many zeros (especially at high frequencies), which zlib compresses very efficiently.

3. Experimental Analysis

Overall pipeline results

Running the full pipeline on the flower_cif video (flower field with windmill) at quality=50, GOP=5, search window= ± 8 px, we obtained the following results:

Metric	Value
Total frames	~250 frames
I-frames	~50 (one every 5 frames)
P-frames	~200
Original raw size	~37,125 KB (uncompressed YCbCr)
Compressed size	~7,053 KB
Compression ratio	~5.39×

The compression ratio is slightly lower than a B-frame design because P-frames are less efficient than B-frames — they only predict from one reference instead of two.

Compression ratio vs. Quality Factor

We ran the I-frame encoder on a single frame across quality factors from 5 to 95 and measured both the compression ratio.

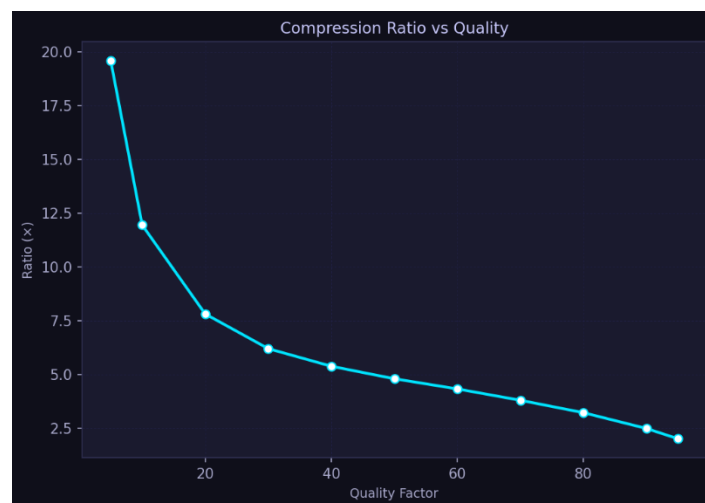


Figure 1 — Compression ratio as a function of the quality factor.

The plot shows that the compression ratio drops sharply as quality increases: at quality=5 we get nearly 20× compression, while at quality=95 we are only around

2×. This makes sense — a higher quality factor means smaller quantisation divisors, which force fewer coefficients to zero, leaving more data to store.

Effect of GOP size on compression

I-frames are larger than P-frames because they carry all information independently. A smaller GOP means more I-frames, which increases file size. A larger GOP means more P-frames but also means that a single corrupted frame can affect more subsequent frames during decoding.

GOP size	I-frame frequency	Expected compression	Robustness
4	Every 4 frames	5.3	High
5	Every 5 frames	5.39	Medium
24	Every 24 frames	5.61	Lower

We chose GOP=5 as a good default. It gives a good balance between file size and error resilience. For a slow-motion flower video with little scene change, a larger GOP would also work well; for a fast-action video with many cuts, a smaller GOP would be safer.

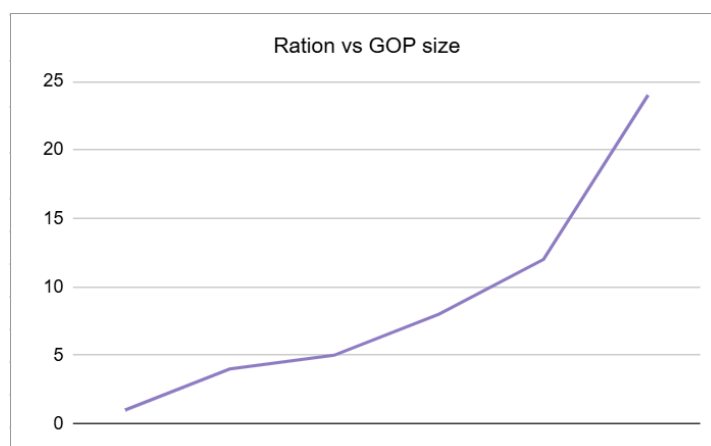


Figure 2 — Compression ratio as a function of the GOP size.

Full pipeline visualisation

The figure below shows the complete pipeline output: original frames, YCbCr channels, one 8×8 DCT block going through all three steps, motion vectors on a P-frame, a residual heat map, and size comparison.

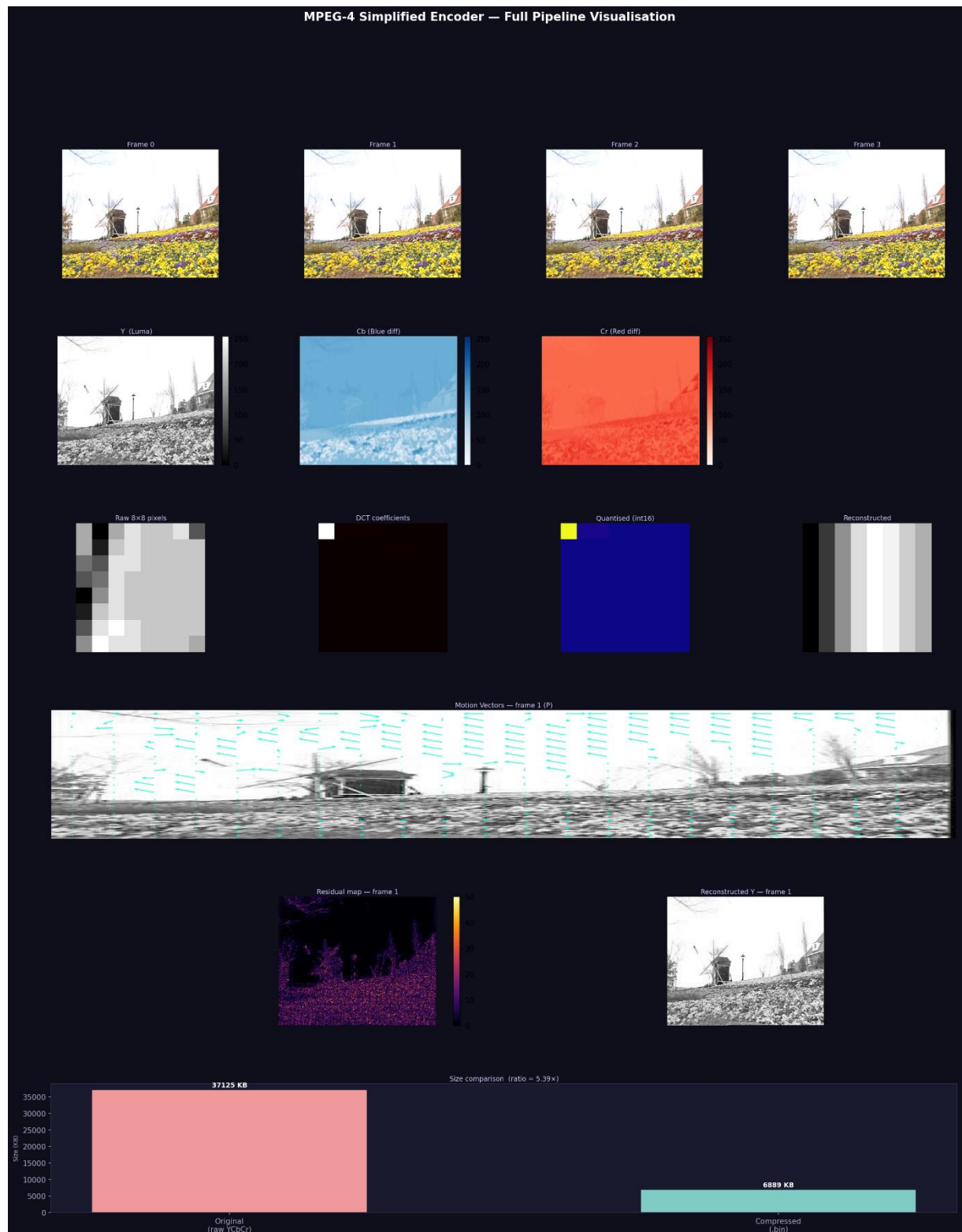


Figure 3— Full pipeline visualisation. Top: original frames. Row 2: Y/Cb/Cr channels. Row 3: 8×8 block → DCT → quantised → reconstructed. Row 4: motion vectors on a P-frame. Row 5: residual map and reconstructed Y. Bottom: size comparison.

Looking at the motion vector plot, we can see that most vectors point in a consistent leftward direction, reflecting the slow pan of the camera across the flower field. The residual map shows higher error around the windmill blades and flower edges — areas with fine detail that are harder to predict from the reference frame.

4. Conclusion

We successfully implemented a full simplified MPEG-4-like video encoder in Python, covering all five stages of the pipeline: pre-processing, I-frame coding, inter-frame coding (I and P frames), entropy coding, and evaluation.

The pipeline achieves a compression ratio of around 5× at quality=50 which is a solid result for a simplified codec without advanced entropy coding. Adding proper run-length encoding on the zigzag-scanned DCT coefficients before the zlib step would further improve compression.

Using only I and P frames keeps the design clean and straightforward: frames are always processed in display order, the encoder is single-pass, and the decoder never needs to buffer future frames. The main trade-off versus a design with B-frames is a slightly lower compression ratio, since P-frames predict only from one past reference instead of two.

The main limitation is speed: our block matching loop is written in pure Python with NumPy and is slow for large numbers of frames. A real codec would use optimised C routines or GPU acceleration for both DCT and motion search.

Overall, the project gave us a concrete understanding of how the core ideas behind JPEG, MPEG, and H.264 actually work — transform coding, quantisation, motion compensation, and lossless compression — by building each piece from scratch.